

DEVOPS MASTER COURSE

Volume 10A

CI/CD Foundations & Pipeline Engineering

*Continuous Integration, Continuous Delivery, Build Systems, Testing, Artifacts, Deployment
Safety, and Delivery Metrics*

Chapters 1-10 | 2026 Edition

From source commit to independently verified production release

This volume treats CI/CD as a production control system. Every lab requires evidence for source, identity, build inputs, test outcomes, artifact identity, approval, deployment, verification, and recovery.

DevOps Master Course

Volume 10 Roadmap

Volume 10 contains six parts. Each part builds from vendor-neutral delivery principles toward enterprise-grade platforms, cloud-native GitOps, security, governance, projects, and interview mastery.

Part	Coverage	Status
10A	CI/CD Foundations and Pipeline Engineering	Current
10B	Jenkins Administration, Declarative Pipelines, Agents, Shared Libraries, and Enterprise Operations	Planned
10C	GitHub Actions, GitLab CI/CD, Reusable Workflows, Runners, and Platform Automation	Planned
10D	Kubernetes Delivery, GitOps, Argo CD, Flux, Progressive Delivery, and Release Engineering	Planned
10E	DevSecOps, Supply-Chain Security, Policy, Observability, Governance, and FinOps	Planned
10F	Enterprise Projects, Troubleshooting, Certifications, Interviews, and Final Capstone	Planned

Part A Learning Outcomes

- Design a CI/CD operating model that reduces batch size while preserving control, evidence, and recoverability.
- Build a multi-stage pipeline with deterministic inputs, isolated execution, caching, quality gates, artifact promotion, and deployment verification.
- Choose a Git integration strategy that supports rapid feedback, protected branches, peer review, and emergency change handling.
- Implement test layers, deployment strategies, environment controls, secret boundaries, and release rollback or fix-forward procedures.
- Measure delivery performance with flow, stability, reliability, queue, and recovery indicators without turning metrics into individual productivity scores.
- Complete a vendor-neutral capstone that can be implemented later in Jenkins, GitHub Actions, GitLab CI/CD, or cloud-native systems.

How to Use This Book

- Read the reason for each control before copying a configuration. CI/CD failures are usually design failures, not YAML syntax failures.
- Keep every lab in version control. Tag the final evidence commit and record the artifact digest produced by that commit.
- Use disposable infrastructure or local containers. Never run destructive deployment experiments against an unapproved production environment.
- Capture both success and failure paths. A pipeline is not production-ready until you have intentionally broken and recovered it.
- Replace example commands with versions supported by your selected platform and confirm current official documentation before production use.

Lab Safety and Evidence Standard

- Use least-privilege identities and short-lived credentials. Do not place tokens, private keys, passwords, or cloud credentials in repositories, build logs, artifacts, screenshots, or cache archives.
- Pin or record toolchain, dependency, action, image, and runner versions. Preserve lock files and checksums whenever the ecosystem supports them.
- Keep build and deployment duties separate for high-risk environments. The identity that compiles an artifact should not automatically receive unrestricted production access.
- Promote one immutable artifact through environments. Do not rebuild source independently for test, staging, and production.
- Retain source revision, pipeline definition revision, runner identity, dependency lock, test results, scan results, artifact digest, approval, deployment record, verification result, and recovery evidence.
- Sanitize retained evidence before publishing a portfolio. Logs, manifests, SBOMs, plans, and test reports can expose internal paths, package names, endpoints, and account identifiers.

Chapter 1: Software Delivery, CI/CD, and the Production Operating Model

CI/CD is not a tool installation. It is an operating model that converts small, reviewed changes into releasable software through repeatable automation, explicit controls, fast feedback, and observable recovery.

Continuous integration reduces integration risk; continuous delivery keeps software deployable; continuous deployment automatically releases changes that satisfy the complete control system.

Learning Objectives

- Differentiate continuous integration, continuous delivery, and continuous deployment.
- Map the end-to-end value stream from idea to verified production behavior.
- Define ownership for source, pipeline, artifact, environment, release, and service health.
- Design feedback loops that reduce batch size and expose risk early.

Core Concepts

Continuous integration: Developers integrate small changes frequently into a shared branch. Automated build and test feedback detects incompatibility while the change context is still fresh.

Continuous delivery: Every accepted change is automatically built, tested, packaged, and kept deployable. Production release may still require a business or risk decision.

Continuous deployment: A qualified change is released automatically without a manual production gate. This requires strong test confidence, safe deployment mechanisms, observability, and rapid recovery.

Value-stream view: Measure waiting, handoffs, rework, queue time, and failure recovery from commit to customer outcome. Optimizing a single fast build step does not help if approvals or environments wait for days.

Separation of concerns: Source control records intent, CI establishes artifact trust, CD promotes an approved immutable artifact, and runtime observability proves whether the service is healthy.

Delivery as a control system: Inputs, policy gates, execution identity, artifacts, approvals, deployment, verification, and recovery form a closed loop. A green job alone is not production evidence.

Reference Pipeline or Command Pattern

```
delivery_contract:
  source_revision: required
  pipeline_revision: required
  build_identity: short_lived
  artifact: immutable_digest
  tests: [unit, integration, policy]
  production_gate: risk_based
  verification: [health, error_rate, business_signal]
  recovery: [rollback_or_fix_forward, incident_record]
```

Production Checklist

- The organization agrees on CI, delivery, and deployment terminology.
- Every pipeline stage has an owner and measurable completion condition.
- The artifact built from a commit is identifiable and immutable.
- Production verification includes service and business signals.
- A tested recovery path exists for deployment and pipeline failure.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Pipeline is fast but releases remain slow	Waiting occurs outside automation	Map queues, approvals, environment readiness, and handoffs; optimize total lead time	Value-stream map, timestamps, queue metrics
Teams call scheduled scripts CI/CD	Integration and release feedback are incomplete	Define required triggers, tests, artifact identity, promotion, and verification	Pipeline contract, acceptance criteria
Production deploy succeeds but users fail	Tool completion is treated as service health	Add post-deploy probes, telemetry analysis, and business transaction checks	Deployment record, dashboards, synthetic result
Every change needs emergency approval	Risk model is not differentiated	Automate low-risk controls and reserve manual decisions for exceptional risk	Policy matrix, approval history, failure data

Hands-on Lab 1: Map and Redesign a Software Delivery Value Stream

Prerequisites

- A sample application repository or a realistic delivery process.
- Access to recent change timestamps, even if simulated.
- A diagram or spreadsheet tool.

Implementation

1. List every step from code completion to production verification, including waits, handoffs, rework, and approvals.
2. Record owner, input, output, average work time, average queue time, failure mode, and retained evidence for each step.
3. Classify activities as value-adding, necessary control, avoidable waste, or unknown.
4. Design a target flow with smaller changes, automated feedback, immutable artifact promotion, and independent verification.
5. Define three flow metrics and three stability or recovery metrics.
6. Create a 30-day improvement backlog with owners and measurable acceptance criteria.

Validation Evidence

- The current-state map includes waiting time, not only execution time.
- The target state has one traceable artifact from build through production.
- Manual gates have explicit risk reasons and required evidence.
- Metrics measure system performance rather than individual developer output.

Cleanup

- Remove confidential names and production identifiers from portfolio diagrams.

- Archive the baseline and target-state evidence with a versioned improvement backlog.

Interview Questions and Model Answers

1. What is the difference between continuous delivery and continuous deployment?

Continuous delivery keeps every accepted change deployable but may require a release decision. Continuous deployment automatically releases every change that satisfies all controls.

2. Why are small batches important?

They reduce merge complexity, narrow failure scope, improve review quality, and make rollback or diagnosis easier.

3. Who owns deployment success?

The service-owning team remains accountable for customer outcomes; the platform team owns the delivery capability and its service levels.

4. Why is a green pipeline insufficient?

It proves configured steps completed, not that tests were complete, the artifact is trusted, or production behavior is healthy.

Chapter Summary

- CI/CD is an operating model, not a product name.
- Optimize the complete value stream and feedback loop.
- Promote immutable artifacts and verify runtime outcomes.
- Control depth should follow risk, with explicit recovery.

Chapter 2: Git Workflows, Change Integration, and Repository Controls

The source repository is the first control plane of software delivery. Branching strategy, commit quality, pull-request policy, ownership, merge rules, and emergency changes determine how quickly a team can integrate while preserving traceability and review.

Learning Objectives

- Compare trunk-based, short-lived feature branch, release branch, and fork workflows.
- Implement protected integration branches and required status checks.
- Design commit, pull-request, ownership, and emergency-change policies.
- Reduce merge queues and long-lived branch divergence.

Core Concepts

Trunk-based development: Developers integrate to one main line frequently, using very short-lived branches or direct commits under strong automated controls. Incomplete behavior is hidden with compatible changes or feature flags.

Feature branches: Short-lived branches isolate review work. They become harmful when they survive for days or weeks, accumulate broad scope, and defer integration.

Release branches: Useful when a supported release line needs independent stabilization or patching. They create merge-forward obligations and should not become the normal path for every environment.

Protected integration: Require review, passing checks, current base, signed or attributable commits where appropriate, and restricted bypass. Rules should be centrally managed and auditable.

Change quality: A change should be small, cohesive, testable, reversible, and linked to intent. Commit messages and pull requests explain why, risk, test evidence, rollout, and recovery.

Emergency path: Define who can bypass, what evidence is required, how the change is limited, and when retrospective review and normal-path reconciliation must occur.

Reference Pipeline or Command Pattern

```
# example repository policy
main:
  require_pull_request: true
  required_reviews: 1
  require_code_owner_for: [pipeline, security, production]
  require_status_checks: [lint, unit, integration, policy]
  require_up_to_date: true
  restrict_force_push: true
  emergency_bypass:
    group: incident_commanders
  retrospective_hours: 24
```

Production Checklist

- Branches are short-lived and changes remain small.
- Main or trunk is protected by review and automated checks.
- Pipeline and deployment configuration has explicit ownership.
- Merge strategy preserves required traceability.
- Emergency bypass is restricted, logged, and retrospectively reviewed.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Frequent merge conflicts	Branches live too long or changes are too broad	Reduce branch lifetime, integrate daily, and split work behind compatibility layers	Branch age, diff size, conflict history
Checks pass before merge but main fails	Branch was tested against stale base	Require current base or merge-queue validation	Merge event, check SHA, queue log
Administrators bypass controls routinely	Rules create friction or ownership is unclear	Fix unreliable checks, define risk tiers, and restrict bypass membership	Bypass audit, check reliability, exception record
Hotfix is lost from future release	Release branch was not merged forward	Automate or track forward-port obligations	Hotfix commit, merge-forward PR, release notes

Hands-on Lab 2: Implement a Protected Integration Workflow

Prerequisites

- A Git repository with at least two contributors or simulated reviewer identities.
- A simple lint or test command.
- Administrative access to repository rules in a lab repository.

Implementation

1. Create a small application and a pipeline check that fails on a deliberate defect.
2. Protect the integration branch with pull-request review and required status checks.
3. Add ownership rules for pipeline and production deployment paths.
4. Create a short-lived branch, open a pull request, observe the failing gate, correct the defect, and merge.
5. Simulate a stale branch and validate the selected update or merge-queue policy.
6. Document the emergency-change path and perform a tabletop review without bypassing controls.

Validation Evidence

- Unreviewed changes cannot merge through the normal path.
- The required check is evaluated for the exact merge candidate.
- Pipeline configuration changes require the designated owner.
- The repository records reviewer, checks, merge revision, and exception policy.

Cleanup

- Remove test bypass permissions and temporary accounts.
- Retain sanitized screenshots or exported rule configuration as evidence.

Interview Questions and Model Answers

1. When is trunk-based development appropriate?

When teams can integrate frequently, maintain fast reliable tests, use compatible changes or flags, and keep trunk releasable.

2. Why are long-lived branches risky?

They defer integration, hide incompatibilities, increase conflict scope, and create large hard-to-review releases.

3. What should a pull request explain beyond code?

Intent, risk, test evidence, rollout method, compatibility, data impact, and recovery.

4. Should administrators bypass branch rules?

Only through a narrow emergency process with strong logging, limited scope, and rapid retrospective reconciliation.

Chapter Summary

- Git policy is part of the delivery control plane.
- Prefer frequent integration and small reviewable changes.
- Protect the exact merge candidate with reliable checks.
- Emergency paths must remain exceptional and auditable.

Chapter 3: Pipeline Architecture, Stages, Jobs, Runners, and Execution Isolation

A pipeline is a directed workflow executed by identities on compute. Good architecture separates orchestration from execution, makes dependencies explicit, keeps jobs reproducible, limits privilege, and preserves evidence across parallel and conditional paths.

Learning Objectives

- Design stage and job boundaries around evidence and failure isolation.
- Use directed acyclic graphs, parallelism, matrices, conditions, and reusable pipeline components.
- Select hosted, self-hosted, ephemeral, and isolated runner patterns.
- Control concurrency, cancellation, timeouts, retries, and resource usage.

Core Concepts

Workflow graph: Jobs should declare real dependencies rather than rely on accidental stage order. Independent checks can run in parallel; promotion depends on complete evidence.

Job isolation: Each job begins from a known environment, receives minimum inputs, and publishes explicit outputs. Hidden shared workspaces create non-reproducible coupling.

Runner trust: Hosted runners reduce maintenance; self-hosted runners enable network or hardware access but expand patching and persistence risk. Ephemeral runners reduce cross-job contamination.

Reusable components: Central templates or reusable workflows standardize controls. Pin versions and define inputs, outputs, permissions, and compatibility just like a software API.

Concurrency control: Prevent overlapping production deployments or stale pipeline completion. Use environment locks, cancellation of superseded runs, bounded parallelism, and timeouts.

Failure semantics: Retries suit transient failures, not deterministic defects. Jobs must distinguish failed, cancelled, skipped, unstable, and manually stopped outcomes.

Reference Pipeline or Command Pattern

```
name: reference-ci
on: [pull_request]
permissions:
  contents: read
jobs:
  validate:
    runs-on: ephemeral-linux
    timeout-minutes: 10
    steps: [checkout, lint, unit-test]
  integration:
    needs: validate
    strategy:
      matrix: {runtime: [current, previous]}
    steps: [checkout, integration-test]
  package:
    needs: integration
    outputs: {artifact_digest: sha256}
    steps: [build, generate-sbom, publish]
```

Production Checklist

- Dependencies are explicit and independent jobs run in parallel.
- Runners are patched, isolated, and assigned minimum network access.
- Workflow components are pinned and versioned.
- Timeout, retry, cancellation, and concurrency behavior is defined.
- Artifacts and reports are passed explicitly, not through hidden local state.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Job passes only after another ran first	Hidden shared workspace or undeclared dependency	Use isolated workspaces and publish/download explicit artifacts	Runner filesystem, job graph, artifact log
Old commit deploys after newer commit	No concurrency or supersession rule	Serialize environment deployment and cancel stale queued work	Run timestamps, environment lock history
Self-hosted runner leaks credentials	Persistent machine retains workspace or process state	Use ephemeral runners, cleanup verification, and scoped credentials	Runner image, process audit, credential issuance
Pipeline retries mask defects	Retry applied to deterministic failure	Classify transient dependencies and cap retries with backoff	Attempt logs, error classification, dependency status

Hands-on Lab 3: Build a Parallel, Isolated Multi-Stage Pipeline

Prerequisites

- A repository and CI platform supporting multiple jobs.
- A small application with lint, unit, and integration commands.
- A safe artifact store or workflow artifact feature.

Implementation

1. Create separate validate, unit, integration, package, and evidence jobs.
2. Run lint and unit tests in parallel, then gate integration on both.
3. Use a matrix to test two supported runtime versions.
4. Publish test reports between jobs rather than sharing a persistent directory.
5. Add timeouts, cancellation of superseded pull-request runs, and a concurrency lock for a simulated environment.
6. Intentionally create a hidden workspace dependency, observe failure on an isolated runner, then correct it with explicit artifacts.

Validation Evidence

- The job graph shows parallel independent work and explicit dependencies.
- Each job succeeds on a fresh runner.
- A stale run cannot overtake a newer environment deployment.
- Published reports and artifact metadata identify the source revision.

Cleanup

- Delete temporary self-hosted runners or revoke their registration tokens.
- Remove lab artifacts according to retention policy.

Interview Questions and Model Answers

1. Why use a DAG rather than only sequential stages?

A DAG exposes true dependencies, shortens feedback by parallelizing independent work, and makes evidence flow explicit.

2. What is the main risk of persistent self-hosted runners?

Cross-job contamination through files, processes, credentials, caches, or modified tooling.

3. When should a job retry?

Only for classified transient failures with bounded attempts and backoff, never to hide deterministic test or policy defects.

4. Why pin reusable workflow versions?

An unpinned dependency can change pipeline behavior without a source change in the consuming repository.

Chapter Summary

- Pipelines are execution graphs, not long shell scripts.
- Isolate jobs and pass outputs explicitly.
- Runner trust and privilege are core security decisions.
- Concurrency and failure semantics prevent stale or ambiguous delivery.

Chapter 4: Build Engineering, Dependency Control, Caching, and Reproducibility

The build converts source into a deployable artifact. A trustworthy build has declared inputs, locked dependencies, a controlled toolchain, deterministic or explainable outputs, safe caching, and provenance that connects the artifact to its source and builder.

Learning Objectives

- Design hermetic or tightly controlled build inputs.
- Use dependency locks, checksums, mirrors, and update automation.
- Optimize caches without making them a source of hidden correctness or security risk.
- Generate artifact digests, SBOMs, and build provenance.

Core Concepts

Declared inputs: Source, toolchain, dependencies, build arguments, platform, and configuration must be identifiable. Network access during builds should be deliberate and minimized.

Dependency locking: Lock files and checksums make resolution repeatable. Version ranges alone allow different builds from the same commit.

Reproducibility: Bit-for-bit identity is ideal but not always practical. At minimum, explain non-deterministic fields and prove that a controlled rebuild yields equivalent functionality and dependency graph.

Cache design: Caches accelerate dependency downloads and compilation. Scope keys by platform and lock digest, prevent untrusted branches from poisoning privileged caches, and treat cache misses as valid.

Container builds: Multi-stage builds separate compilation from runtime, reducing final contents. Pin base images by digest for controlled releases and rebuild regularly for patched dependencies.

Build provenance: Record where, when, how, and from which inputs an artifact was produced. Provenance supports verification but does not prove the source itself is safe.

Reference Pipeline or Command Pattern

```
# reproducible build outline
SOURCE_SHA=$(git rev-parse HEAD)
LOCK_SHA=$(sha256sum dependency.lock | cut -d' ' -f1)

build --clean --locked --source "$SOURCE_SHA"
generate-sbom --output sbom.spdx.json
DIGEST=$(sha256sum dist/app.tar.gz | cut -d' ' -f1)
create-provenance --source "$SOURCE_SHA" --lock "$LOCK_SHA" --artifact "$DIGEST"
```

Production Checklist

- Dependencies are locked and integrity-checked.
- Build toolchain and base images are versioned or digest-pinned.
- Untrusted changes cannot poison protected caches.
- The build succeeds without undocumented local files.
- Artifact digest, SBOM, and provenance are retained.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Same commit creates different dependency tree	Resolver uses floating ranges or missing lock file	Require locked install and validate lock changes in review	Lock file, resolver log, dependency manifest
Build works only on developer laptop	Undeclared local tools or files	Build in a clean container or ephemeral runner with explicit toolchain	Runner image digest, build manifest
Cache causes mysterious stale output	Cache key omits relevant input	Key cache from lock, platform, compiler, and build mode; add clean-build comparison	Cache key, hit log, clean-build result
Small image still contains secrets	Secret copied into layer or build context	Use secret mounts, minimal context, and image-layer inspection	Build definition, layer history, secret scan

Hands-on Lab 4: Create a Reproducible Build with SBOM and Provenance

Prerequisites

- A small compiled or packaged application.
- A package manager with lock-file support.
- Container or clean-runner capability and checksum utilities.

Implementation

1. Pin the runtime or compiler and create or refresh the dependency lock file.
2. Build from a clean environment using locked dependency resolution.
3. Produce the deployable archive or container image and record its digest.
4. Generate an SBOM and a provenance statement containing source revision, builder, toolchain, and dependency lock digest.
5. Repeat the build on a second clean runner and compare artifact or normalized-content results.
6. Add a cache keyed by platform and lock digest, then prove a cache miss still succeeds.

Validation Evidence

- The same source and lock resolve the same dependency graph.
- No undeclared developer files are required.
- The artifact digest and SBOM are linked to the pipeline run.
- Protected builds do not restore caches written by untrusted code.

Cleanup

- Delete temporary build secrets and runner workspaces.
- Apply artifact and cache retention policies while retaining required evidence.

Interview Questions and Model Answers

1. Why is a lock file important?

It records exact dependency resolution so the same source does not silently build against a different graph.

2. Is a reproducible build automatically secure?

No. It improves integrity and investigation, but reproducibly building vulnerable or malicious source remains unsafe.

3. Why should base images be pinned by digest?

Tags are mutable; a digest identifies the exact content used for a release.

4. What is the danger of shared caches?

Untrusted jobs can introduce poisoned dependencies or outputs that privileged jobs later consume.

Chapter Summary

- A release artifact must be traceable to declared inputs.
- Lock dependencies and control toolchains.
- Caches improve speed but must not become hidden state.
- Digests, SBOMs, and provenance strengthen artifact verification.

Chapter 5: Automated Testing, Quality Gates, and Test Evidence

Testing in CI/CD is a risk model expressed as executable evidence. Effective pipelines combine fast deterministic checks with progressively broader tests, manage flaky behavior explicitly, and gate promotion on results that are relevant to the changed system.

Learning Objectives

- Design a layered test portfolio for speed, confidence, and maintainability.
- Separate unit, component, contract, integration, end-to-end, performance, security, and resilience tests.
- Manage flaky tests without normalizing reruns.
- Publish machine-readable reports and risk-based quality gates.

Core Concepts

Test pyramid and portfolio: Run many fast isolated tests, fewer integration tests, and a small number of expensive end-to-end scenarios. The right shape depends on architecture and failure risk.

Contract testing: Validate interfaces between independently deployed components without requiring the complete production topology. Contracts must be versioned and owned.

Test data: Use deterministic factories, isolated namespaces, and safe synthetic data. Shared mutable datasets create order dependence and privacy risk.

Flaky tests: A flaky test produces inconsistent outcomes without relevant code change. Quarantine may protect flow temporarily, but the test needs an owner, deadline, and visible risk treatment.

Quality gates: Gates should use meaningful thresholds, changed-code scope, critical path, and trend. A single coverage percentage is not a substitute for risk analysis.

Evidence: Publish structured test results, coverage, logs, environment identity, dependency versions, and failure artifacts with retention appropriate to audit and diagnosis.

Reference Pipeline or Command Pattern

```
test_stages:
  fast: [format, lint, unit]
  component: [service, contract]
  integration: [database, queue, external_stub]
  release: [smoke, security, performance_budget]
quality_gate:
  critical_tests: must_pass
  new_code_coverage: ">= 80%"
  flaky_quarantine: visible_and_expiring
reports: [junit, coverage, logs]
```

Production Checklist

- Fast checks complete early and fail clearly.
- Tests are isolated from order and shared mutable data.
- Critical contracts and migrations are exercised.
- Flaky tests have owners, deadlines, and tracked impact.
- Reports identify source, environment, and tool versions.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Tests pass individually but fail together	Shared state or order dependency	Isolate namespaces, reset state, and randomize order to expose coupling	Seed, test order, environment snapshot
Pipeline reruns until green	Flakiness is treated as normal	Track retry rate, fail or quarantine visibly, and fix root cause	Attempt history, owner, expiry date
High coverage but production defects continue	Coverage does not target risk or behavior	Add boundary, contract, failure, and mutation-oriented tests	Defect analysis, test map, coverage diff
End-to-end suite takes hours	Too much behavior tested only at top layer	Move checks to unit, component, or contract layers and parallelize isolated scenarios	Duration by test, dependency graph

Hands-on Lab 5: Design and Enforce a Layered Test Pipeline

Prerequisites

- A sample application with at least one internal dependency or API.
- A unit-test framework and machine-readable report format.
- A disposable integration dependency such as a local database container.

Implementation

1. Create fast unit tests for core logic and publish a structured report.
2. Add a component or API contract test with deterministic fixtures.
3. Start an isolated database or service dependency and execute integration tests.
4. Introduce one intentionally flaky test, measure inconsistent results, then remove the non-determinism rather than relying on retries.
5. Add a changed-code quality gate and a mandatory critical-test gate.
6. Archive failure diagnostics while redacting secrets and personal data.

Validation Evidence

- Fast tests run before expensive stages.
- Integration tests use isolated, disposable dependencies.
- The pipeline exposes flaky behavior and does not silently rerun to green.
- Reports and gates correspond to the exact source revision.

Cleanup

- Destroy integration containers and test namespaces.
- Remove deliberately flaky code and sensitive failure artifacts.

Interview Questions and Model Answers

1. What makes a quality gate useful?

It is tied to a known risk, uses reliable evidence, and has a clear corrective action rather than an arbitrary number.

2. Why are end-to-end tests insufficient?

They are slow, broad, harder to diagnose, and often fragile; lower layers provide faster, more precise feedback.

3. How should flaky tests be handled?

Make the flakiness visible, assign ownership and deadline, quarantine only when necessary, and measure the lost confidence.

4. What should test evidence identify?

Source revision, test version, environment, dependencies, timing, outcome, and diagnostic artifacts.

Chapter Summary

- Testing is a portfolio of risk controls.
- Use fast lower-level feedback and targeted release tests.
- Do not normalize flaky reruns.
- Quality gates must connect evidence to meaningful risk.

Chapter 6: Artifacts, Versioning, Registries, Promotion, and Retention

The artifact is the unit of promotion. A mature pipeline builds once, assigns immutable identity, stores metadata with the artifact, promotes the same digest through environments, and retains enough evidence to reproduce or investigate every release.

Learning Objectives

- Define artifact identity with versions, digests, and metadata.
- Use registries and repositories as controlled promotion boundaries.
- Apply semantic versioning, release channels, and pre-release identifiers appropriately.
- Design retention, replication, immutability, and rollback access.

Core Concepts

Build once, promote: Testing and production must consume the same artifact bytes. Environment-specific configuration is supplied separately at deployment or runtime.

Version versus digest: A version communicates compatibility and release intent. A cryptographic digest identifies exact content. Use both where possible.

Semantic versioning: For a declared public API, increment major for incompatible change, minor for backward-compatible functionality, and patch for backward-compatible fixes. Internal deployment identifiers may use commit and build metadata.

Registry controls: Separate write and read privileges, scan artifacts, enforce immutability for released versions, replicate critical content, and log promotions and deletions.

Promotion metadata: Record source, build, test, scan, approval, target, deployer, and verification. Promotion changes status or repository location, not artifact contents.

Retention and rollback: Keep artifacts and evidence long enough for operational recovery, legal requirements, and vulnerability response. Test that older artifacts remain deployable with current configuration and dependencies.

Reference Pipeline or Command Pattern

```
artifact:
  name: checkout-api
  version: 2.4.1
  digest: sha256:REPLACE_ME
  source_commit: abc1234
  sbom: sbom.spdx.json
  provenance: provenance.json
promotion:
  from: candidate
  to: production-approved
  required: [tests_passed, scan_passed, approval, signature_verified]
```

Production Checklist

- Released artifact references include immutable digest.
- The same artifact is used in every environment.
- Registry write and deployment-read permissions are separated.
- Promotion events and metadata are auditable.

- Retention supports rollback and vulnerability investigation.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Staging passed but production failed immediately	Production artifact was rebuilt or tag moved	Promote exact digest and verify before deploy	Digest in each environment, registry event
Rollback artifact no longer exists	Retention deleted old release	Align retention with rollback window and pin released versions	Retention policy, release inventory
Version does not communicate compatibility	No public API or inconsistent version policy	Document versioning contract and automate release metadata	Release policy, tags, changelog
Registry compromise changes release tag	Mutable tags and broad write access	Enforce immutability, signatures, digest verification, and restricted promotion identity	Registry audit, signature result, digest

Hands-on Lab 6: Build Once and Promote an Immutable Artifact

Prerequisites

- A package or container registry available for a lab.
- A sample application and build pipeline.
- Ability to create candidate and approved repositories or channels.

Implementation

1. Build one artifact and attach source revision, version, digest, SBOM, and provenance metadata.
2. Publish it to a candidate repository or release channel.
3. Run verification by downloading and checking the exact digest.
4. Promote the same content to an approved channel without rebuilding.
5. Deploy or simulate deployment to two environments using the identical digest and different external configuration.
6. Attempt a tag mutation, confirm the control prevents or detects it, and record the audit evidence.

Validation Evidence

- Candidate and approved references resolve to identical content digest.
- The deployment record contains both version and digest.
- Registry permissions distinguish build publication from production consumption.
- Rollback content remains available and verified.

Cleanup

- Delete only temporary candidate artifacts permitted by policy.
- Keep the final evidence manifest and approved artifact according to lab retention.

Interview Questions and Model Answers

1. Why use both semantic version and digest?

The version communicates compatibility and release intent; the digest proves exact content identity.

2. What does build once, promote mean?

Compile or package once, then move the same immutable content through control states or repositories.

3. Should production deploy by mutable tag?

No. A friendly tag can select a candidate, but the resolved digest should be recorded and verified.

4. What determines artifact retention?

Rollback requirements, incident investigation, vulnerability response, audit, legal constraints, storage cost, and rebuild capability.

Chapter Summary

- The artifact, not the branch, is promoted.
- Versions communicate meaning; digests identify bytes.
- Registry permissions and immutability protect releases.
- Retention and metadata make rollback and investigation possible.

Chapter 7: Environments, Configuration, Secrets, Identity, and Deployment Authorization

CI/CD crosses trust boundaries. Environment topology, configuration management, secret handling, workload identity, approval, and deployment authorization determine whether a pipeline can change only the intended target with the intended privileges.

Learning Objectives

- Design environment boundaries that support realistic validation without unmanaged duplication.
- Separate application artifact from environment configuration.
- Replace long-lived pipeline credentials with short-lived workload identity where supported.
- Implement least-privilege deployment authorization and protected environments.

Core Concepts

Environment purpose: Each environment needs a defined decision: developer feedback, integration, performance, security, release qualification, or production. Duplicate environments without a decision create cost and drift.

Configuration separation: Store non-secret configuration as reviewed declarative data. Inject it independently from the immutable artifact and validate schema and compatibility.

Secret boundary: Use a dedicated secrets system, short-lived retrieval, minimal scope, rotation, and log masking. Masking is not a guarantee that a secret cannot leak into artifacts or subprocess output.

Workload identity: Federate the pipeline runtime to the target platform and issue short-lived credentials based on repository, branch, environment, workflow, or runner claims.

Protected environments: Bind production jobs to approvals, concurrency, allowed branches, deployment windows, and specific credentials. Approval should evaluate evidence, not repeat automated checks manually.

Privilege separation: Build jobs generally need source and registry-write access; deployment jobs need approved artifact-read and target-change access. Avoid one omnipotent credential across all stages.

Reference Pipeline or Command Pattern

```
deployment_identity:
  authentication: workload_federation
  token_ttl_minutes: 15
  claims:
    repository: platform/checkout
    branch: main
    environment: production
  permissions:
    - read: approved-artifact-digest
    - deploy: checkout-production
    - deny: registry-delete
    - deny: unrelated-environments
```

Production Checklist

- Every environment has a purpose, owner, and lifecycle.
- Configuration is versioned and schema-validated.
- Secrets are retrieved at runtime and never stored in artifacts.
- Production identity is short-lived and target-scoped.

- Approvals, concurrency, and allowed sources are enforced by the platform.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Secret appears in log despite masking	Value transformed, split, or printed by tool	Prevent output, use no-debug modes, rotate secret, and inspect artifacts and caches	Redacted logs, rotation event, incident record
Pipeline can deploy to any account	Credential is static and broadly privileged	Use workload identity with claim conditions and target-specific roles	Token claims, role policy, denied test
Staging differs from production	Manual configuration and unmanaged dependencies	Version configuration, automate environment creation, and compare required capabilities	Config revision, dependency inventory
Approval provides no risk information	Gate occurs before evidence is assembled	Present artifact digest, change summary, test/scan results, deployment plan, and recovery	Approval payload, reviewer identity

Hands-on Lab 7: Implement a Protected Environment with Short-Lived Identity

Prerequisites

- A CI platform with environment controls or a simulated policy layer.
- A target platform supporting federated or short-lived credentials, or a local token broker simulation.
- A secrets manager or safe lab substitute.

Implementation

1. Create development and production environment definitions with separate target identifiers.
2. Move environment configuration outside the application artifact and add schema validation.
3. Configure the pipeline to exchange runtime identity for a short-lived, target-scoped credential.
4. Restrict production deployment to the protected branch and approved artifact digest.
5. Add an approval that displays source, tests, artifact digest, deployment plan, and recovery method.
6. Attempt deployment from an unauthorized branch and to an unrelated target; preserve the denied evidence.

Validation Evidence

- No long-lived production secret is stored in repository or pipeline variables.
- Credential claims bind the run to repository, branch, and environment.
- Unauthorized source and target combinations are denied.
- The production job receives only the permissions required for deployment and verification.

Cleanup

- Revoke temporary trust relationships and lab roles.

- Rotate any fallback secret used to simulate identity federation.

Interview Questions and Model Answers

1. Why is masking insufficient secret protection?

Secrets can be transformed, encoded, copied into artifacts, exposed by subprocesses, or available to malicious code before masking.

2. What should an approval review?

The exact artifact, source change, automated evidence, target, deployment method, risk, and recovery.

3. Why separate build and deploy identities?

A compromised build should not automatically gain production control, and deployment should consume only approved artifacts.

4. What is environment parity?

Matching the capabilities and relevant configuration assumptions needed to make validation meaningful, not necessarily identical capacity or cost.

Chapter Summary

- Environment boundaries are security and validation boundaries.
- Keep configuration outside immutable artifacts.
- Prefer short-lived workload identity over static credentials.
- Production gates must bind exact evidence to exact target authorization.

Chapter 8: Deployment Strategies, Database Changes, Feature Flags, and Release Safety

Deployment changes running systems under uncertainty. Rolling, blue-green, canary, and recreate strategies control exposure differently. Safe releases combine compatibility, progressive traffic, database discipline, feature control, automated verification, and a practiced rollback or fix-forward decision.

Learning Objectives

- Select deployment strategy from availability, capacity, state, and risk requirements.
- Implement progressive verification and automated stop conditions.
- Design backward-compatible database and API changes.
- Use feature flags with ownership, security, and retirement controls.

Core Concepts

Rolling deployment: Replaces instances gradually. It is capacity-efficient but temporarily runs mixed versions, requiring backward and forward compatibility.

Blue-green: Maintains old and new environments and switches traffic. Rollback is fast when state and database compatibility permit, but duplicate capacity and data synchronization matter.

Canary: Exposes a small traffic or user segment first, compares health and business signals, then expands or aborts. A canary without automatic analysis is merely a small risky deployment.

Database expand-contract: Add compatible schema first, deploy code that tolerates old and new states, migrate data, switch behavior, and remove obsolete schema only after all consumers are safe.

Feature flags: Decouple code deployment from user exposure. Flags need ownership, targeting controls, audit, tests for important states, and expiry to avoid permanent complexity.

Recovery decision: Rollback is fastest when changes are reversible. Fix forward may be safer for irreversible data changes. Define decision criteria before the incident.

Reference Pipeline or Command Pattern

```
release_strategy:
  type: canary
  steps: [5, 20, 50, 100]
  observation_minutes: [10, 15, 20]
  analysis:
    error_rate: "<= baseline + 0.5%"
    p95_latency: "<= baseline * 1.10"
    checkout_success: ">= baseline - 1%"
  abort: automatic
  database: expand_contract
  feature_flag_expiry_days: 30
```

Production Checklist

- Strategy matches capacity, state, compatibility, and recovery constraints.
- Mixed-version compatibility is tested where required.
- Progressive steps have measurable automated abort conditions.
- Database changes follow expand-contract or equivalent compatibility plan.
- Feature flags have owners, audits, and removal dates.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Rollback fails after schema migration	New schema is not backward compatible	Use expand-contract, backup, tested restore, and fix-forward decision criteria	Migration version, compatibility test, restore drill
Canary appears healthy but users fail	Only infrastructure metrics are checked	Add transaction and business outcome signals segmented by canary cohort	Cohort dashboard, synthetic result
Blue-green switch causes session loss	State is local or not shared compatibly	Externalize or drain sessions and test connection behavior	Session design, drain log, switch test
Hundreds of stale flags remain	No lifecycle ownership or expiry	Create registry, owner, expiry alert, and removal gate	Flag inventory, age report, cleanup PR

Hands-on Lab 8: Execute a Progressive Release with Automated Abort

Prerequisites

- A local load balancer, container platform, or deployment simulator.
- Two application versions, one containing a controlled failure mode.
- Metrics or scripted health and business checks.

Implementation

1. Deploy a stable baseline and record latency, error, and transaction-success signals.
2. Prepare a backward-compatible new version and a separate feature flag.
3. Release to a 5 percent canary cohort and run automated analysis.
4. Activate the controlled defect, verify the release aborts, and confirm stable traffic remains healthy.
5. Correct the defect and progress through 5, 20, 50, and 100 percent stages.
6. Document rollback versus fix-forward criteria and perform one recovery exercise.

Validation Evidence

- Traffic expansion occurs only after passing measured criteria.
- Abort is automatic and limits user exposure.
- Stable and canary telemetry can be compared by version or cohort.
- The database or state model remains compatible during mixed-version operation.

Cleanup

- Remove the test defect and retire temporary feature flags.
- Scale down duplicate environments and retain the release analysis report.

Interview Questions and Model Answers

1. What makes a canary effective?

Limited exposure, comparable telemetry, meaningful service and business signals, automatic analysis, and automatic stop or rollback.

2. Why is database rollback difficult?

Data changes may be irreversible or incompatible with old code, so schema and application evolution must be staged.

3. When is blue-green unsuitable?

When duplicate capacity is impractical, state cannot be synchronized, or database changes prevent old version operation.

4. What is feature-flag debt?

Accumulated stale flags and alternate code paths that increase test combinations, cognitive load, and security risk.

Chapter Summary

- Deployment strategy controls exposure but cannot replace compatibility.
- Progressive delivery needs automated analysis and stop conditions.
- Use expand-contract for shared data and interfaces.
- Flags must be governed and removed.

Chapter 9: Pipeline Observability, DORA Metrics, SLOs, and Recovery Engineering

A delivery platform is a production service. Teams need telemetry for queue time, runner capacity, pipeline reliability, deployment flow, change outcomes, and recovery. Metrics should improve the system, not rank individuals or reward unsafe gaming.

Learning Objectives

- Instrument pipelines with logs, metrics, traces, correlation identifiers, and evidence manifests.
- Measure delivery flow and stability using current DORA-style metrics and supporting diagnostics.
- Define CI/CD platform SLOs and error budgets.
- Design recovery for runner, registry, pipeline, credential, and deployment failures.

Core Concepts

Delivery performance: Track deployment frequency, change lead time, change fail percentage, and failed deployment recovery time. Current DORA guidance also highlights deployment rework rate as an additional signal.

Diagnostic metrics: Queue time, job duration, cache hit rate, flaky-test rate, runner utilization, approval wait, rollback rate, and artifact retrieval errors explain why outcomes change.

Correlation: Carry source revision, pipeline run, artifact digest, deployment, environment, version, and incident identifiers through logs and telemetry.

Pipeline SLO: Examples include time to first feedback, pipeline availability, queue latency, successful artifact publication, and recovery time. Error budgets guide reliability investment.

Failure taxonomy: Separate source defect, flaky test, infrastructure outage, capacity exhaustion, dependency failure, policy rejection, credential failure, and deployment health regression.

Recovery engineering: Back up pipeline definitions through Git, replicate critical artifacts, rebuild ephemeral runners, rotate credentials, test registry restore, and maintain a manual but controlled release path for platform outage.

Reference Pipeline or Command Pattern

```
pipeline_slos:
  first_feedback_p95_minutes: 8
  main_pipeline_success_excluding_code_defects: 99.5%
  runner_queue_p95_minutes: 3
  artifact_publish_success: 99.9%
  platform_recovery_minutes: 60
correlation:
  fields: [commit_sha, run_id, artifact_digest, deployment_id, environment, incident_id]
```

Production Checklist

- Metrics distinguish code failures from platform failures.
- Queue and duration are measured by stage and runner class.
- DORA metrics are defined consistently for the service.
- Pipeline SLOs have owners and error-budget actions.
- Critical artifacts, credentials, and execution capability have tested recovery.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Success rate falls but cause is unknown	Failures are not classified	Add structured outcome taxonomy and dependency status	Run metadata, error class, dependency health
Teams optimize deployment frequency recklessly	Metric is used as target without stability context	Balance flow with failure, recovery, reliability, and customer outcomes	Metric review, incident trend, SLO impact
Queue time spikes unpredictably	Runner capacity and job demand are invisible	Measure arrival rate, service time, utilization, and autoscaling response	Queue series, runner metrics, scaling events
Pipeline outage stops emergency release	No tested degraded-mode process	Maintain controlled break-glass path and regularly exercise it	Runbook, exercise log, access audit

Hands-on Lab 9: Create a CI/CD Observability and Recovery Dashboard

Prerequisites

- Pipeline run data from prior labs or a generated sample dataset.
- A metrics and dashboard tool, spreadsheet, or local observability stack.
- A documented pipeline failure scenario.

Implementation

1. Add structured fields for source, run, artifact, environment, and deployment identifiers.
2. Classify ten sample outcomes into source, test, platform, dependency, policy, credential, or deployment failures.
3. Calculate first-feedback time, queue time, total lead time, deployment frequency, change fail percentage, and recovery time.
4. Define three pipeline SLOs and an error-budget response.
5. Simulate runner unavailability or registry outage and execute the recovery runbook.
6. Create a review note explaining which improvement is supported by data and which conclusion would be unsafe.

Validation Evidence

- A release can be traced from commit to artifact to deployment and incident.
- Outcome metrics exclude or classify non-platform code defects appropriately.
- The dashboard shows both flow and stability.
- Recovery objectives are tested rather than assumed.

Cleanup

- Remove internal identifiers from published dashboard evidence.
- Close temporary break-glass access and record its audit trail.

Interview Questions and Model Answers

1. What are the core delivery performance metrics?

Deployment frequency, change lead time, change fail percentage, and failed deployment recovery time; current DORA materials also discuss deployment rework rate.

2. Why not compare individual developers with DORA metrics?

They describe a socio-technical delivery system; individual ranking encourages gaming and damages collaboration and quality.

3. What should a pipeline SLO measure?

User-facing platform reliability such as feedback latency, queue time, availability, artifact publication, or recovery.

4. Why classify failures?

A code defect, flaky test, runner outage, or credential failure requires different ownership and corrective action.

Chapter Summary

- Observe CI/CD as a production service.
- Balance delivery flow with stability and recovery.
- Use correlation identifiers across source, artifact, deploy, and incident.
- Test degraded-mode and control-plane recovery.

Chapter 10: CI/CD Foundation Architecture and Integrated Capstone

The foundation capstone combines Git governance, isolated pipeline execution, reproducible build, layered testing, immutable artifact promotion, protected deployment identity, progressive release, observability, and recovery into one coherent delivery system.

Learning Objectives

- Design a vendor-neutral CI/CD reference architecture.
- Implement a complete source-to-production evidence chain.
- Exercise failure, rollback, and platform recovery paths.
- Defend architecture trade-offs through a production-readiness review.

Core Concepts

Reference architecture: Source repository, identity provider, runner service, dependency proxy, artifact registry, policy engine, target environment, secrets system, telemetry backend, and evidence store form distinct trust zones.

Pipeline contract: Every stage defines inputs, outputs, identity, permissions, evidence, failure semantics, timeout, retry, and recovery. Reusable components implement the contract consistently.

Promotion model: Pull-request CI produces evidence; main integration produces a candidate; release qualification approves exact digest; CD deploys progressively; verification records outcome.

Independent verification: Deployment scripts should not be the only source of truth. External health probes and telemetry confirm the target state and detect silent failure.

Threat and failure model: Consider malicious pull request, compromised dependency, poisoned cache, stolen runner token, mutable artifact, broad deployment role, registry outage, failed migration, and telemetry blind spot.

Production readiness: Review reliability, security, cost, performance, operability, compliance, evidence, ownership, and recovery before declaring the system complete.

Reference Pipeline or Command Pattern

```
pipeline:
  pull_request: [lint, unit, contract, policy]
  main: [clean_build, sbom, provenance, integration, publish_candidate]
  release: [verify_digest, approve, promote]
  production: [short_lived_identity, canary, automated_analysis, full_rollout]
  post_deploy: [synthetic, slo_check, evidence_manifest]
recovery:
  app: rollback_or_fix_forward
  platform: rebuild_runner_and_restore_registry_access
```

Production Checklist

- Every release maps to reviewed source and exact artifact digest.
- Build, test, promotion, deployment, and verification identities are least-privileged.
- The pipeline produces structured test, scan, SBOM, provenance, approval, and deployment evidence.
- Progressive release has automatic abort conditions.
- Application and CI/CD platform recovery paths have been exercised.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Capstone works only in happy path	Failure exercises were omitted	Inject dependency, test, runner, registry, identity, and deployment failures	Game-day report, recovery timestamps
Evidence cannot prove exact release	Version, digest, and run identifiers are inconsistent	Create one evidence manifest and propagate correlation fields	Manifest, registry metadata, deploy record
Reusable template change breaks many repositories	No compatibility or staged rollout	Version templates, add contract tests, canary consumers, and deprecation policy	Template release, consumer test results
Manual recovery requires unknown expert	Runbook and access are incomplete	Execute clean-room recovery with a second operator	Runbook revision, access test, retrospective

Hands-on Lab 10: Build, Break, Recover, and Defend a Complete CI/CD System

Prerequisites

- A repository, CI platform, artifact store, and disposable deployment target.
- A sample service with tests and a controllable failure mode.
- A secrets or identity mechanism and a dashboard or evidence store.

Implementation

1. Create the architecture diagram, trust boundaries, ownership matrix, delivery contract, and threat model.
2. Implement protected Git integration and pull-request checks.
3. Build in an isolated runner using locked dependencies; publish digest, SBOM, provenance, and test reports.
4. Promote the same artifact through candidate and approved states.
5. Deploy with short-lived target-scoped identity using a canary and automated analysis.
6. Produce an evidence manifest linking source, pipeline, artifact, approval, deployment, verification, and recovery.
7. Inject at least four failures: poisoned or invalid dependency, flaky test, runner outage, and unhealthy canary.
8. Recover with a second operator following only the runbook.
9. Present the design, measured performance, residual risks, cost model, and 90-day improvement backlog.

Validation Evidence

- A reviewer can trace the production version to exact source and build inputs.
- The same digest is promoted and deployed across environments.
- Unauthorized source, identity, or target combinations fail closed.
- Injected failures are detected, contained, and recovered within documented objectives.

- The final review includes limitations and residual risk, not only successes.

Cleanup

- Destroy disposable infrastructure and revoke all lab trust relationships.
- Retain sanitized source, diagrams, evidence manifest, dashboards, game-day results, and retrospective.

Interview Questions and Model Answers

1. What is the most important capstone evidence?

A traceable chain from reviewed source through exact artifact and authorization to verified production outcome and recovery.

2. Why must another operator execute recovery?

It tests whether knowledge is institutionalized in runbooks and access, rather than held by the original builder.

3. What should be versioned besides application code?

Pipeline definitions, reusable components, policies, configuration schemas, deployment manifests, tests, and runbooks.

4. When is a CI/CD platform production-ready?

When normal delivery, failure containment, recovery, observability, ownership, security, and evidence have all been demonstrated under realistic conditions.

Chapter Summary

- A CI/CD platform is a traceable, least-privilege production control system.
- Build once, test deeply, promote exact content, deploy progressively, and verify independently.
- Failure injection and second-operator recovery prove operability.
- The capstone becomes the baseline for tool-specific Parts B-F.

Appendix A: Vendor-Neutral Pipeline Design Worksheet

Design Area	Required Decision	Evidence
Source integration	Branching model, review, ownership, merge candidate validation	Rules export, sample pull request
Execution	Runner class, isolation, network, timeout, concurrency	Runner image, permissions, queue metrics
Build	Toolchain, locks, cache, reproducibility, provenance	Lock digest, clean build, SBOM, provenance
Testing	Layers, data, flake policy, quality gates	Reports, gate configuration, failure example
Artifacts	Version, digest, registry, promotion, retention	Registry metadata, promotion audit
Identity	Build and deploy identities, trust conditions, token lifetime	Claim policy, denied authorization test
Deployment	Strategy, compatibility, analysis, recovery	Deployment record, canary analysis, rollback drill
Observability	Correlation, DORA metrics, SLOs, alerts	Dashboard, SLO, incident record

Appendix B: Pipeline Command and Investigation Reference

Objective	Representative Command or Action
Identify source revision	<code>git rev-parse HEAD</code>
Verify clean working tree	<code>git status --porcelain</code>
Inspect branch divergence	<code>git log --oneline --graph --decorate --all</code>
Calculate file digest	<code>sha256sum artifact-file</code>
Inspect container digest	<code>docker image inspect --format {{.RepoDigests}} image:tag</code>
Clean container build	<code>docker build --no-cache --pull -t app:candidate .</code>
Generate dependency inventory	Use ecosystem lock/report command and retain machine-readable output

Objective	Representative Command or Action
Publish test results	Emit JUnit-compatible XML and coverage report
Trace release	Search by commit SHA, run ID, artifact digest, deployment ID, and environment
Classify failure	Source, test, platform, dependency, policy, credential, deployment, or verification

Appendix C: Production-Readiness Checklist

- Source integration is protected, reviewed, and validated for the exact merge candidate.
- Pipeline definitions and reusable components are versioned, owned, tested, and pinned.
- Jobs run on isolated, patched runners with bounded network and least privilege.
- Dependencies and toolchains are locked or otherwise reproducibly identified.
- Builds produce immutable artifacts with digest, SBOM, provenance, and source metadata.
- Test layers are reliable, isolated, risk-based, and publish structured evidence.
- Protected caches cannot be written by untrusted execution contexts.
- The same artifact digest is promoted through every environment.
- Configuration and secrets are separated from artifact content.
- Production authentication uses short-lived, target-scoped identity where supported.
- Deployment strategy, compatibility, health analysis, and recovery are documented and tested.
- Feature flags and database migrations have lifecycle and compatibility controls.
- Pipeline queue, reliability, delivery flow, change outcomes, and recovery are observable.
- CI/CD control-plane outage and emergency release procedures have been exercised.
- All evidence is retained and sanitized according to policy.

Appendix D: Ten Additional Portfolio Projects

1. Polyglot Monorepo Pipeline

Create change-aware builds and tests for three services with shared libraries, matrix testing, dependency caching, and selective deployment.

- Required evidence: architecture, pipeline source, threat model, test reports, artifact digest, failure exercise, dashboard, cleanup proof, and retrospective.

2. Reusable Pipeline Component Library

Publish versioned lint, test, build, scan, and deploy components with typed inputs, contract tests, compatibility policy, and canary consumers.

- Required evidence: architecture, pipeline source, threat model, test reports, artifact digest, failure exercise, dashboard, cleanup proof, and retrospective.

3. Immutable Container Promotion System

Build one OCI image, attach SBOM and provenance, sign or attest it, promote by digest, and enforce production verification.

- Required evidence: architecture, pipeline source, threat model, test reports, artifact digest, failure exercise, dashboard, cleanup proof, and retrospective.

4. Database Migration Release Pipeline

Implement expand-contract migrations, compatibility tests, backup evidence, rollout sequencing, and rollback/fix-forward decision records.

- Required evidence: architecture, pipeline source, threat model, test reports, artifact digest, failure exercise, dashboard, cleanup proof, and retrospective.

5. Protected Environment and Workload Identity Lab

Use repository and environment claims to obtain short-lived credentials and prove denied unauthorized deployment paths.

- Required evidence: architecture, pipeline source, threat model, test reports, artifact digest, failure exercise, dashboard, cleanup proof, and retrospective.

6. Progressive Delivery Simulator

Implement rolling, blue-green, and canary strategies with automated analysis and compare cost, speed, and failure exposure.

- Required evidence: architecture, pipeline source, threat model, test reports, artifact digest, failure exercise, dashboard, cleanup proof, and retrospective.

7. CI/CD Observability Platform

Correlate source, pipeline, artifact, deployment, and incident data; publish SLOs, queue metrics, DORA metrics, and error budgets.

- Required evidence: architecture, pipeline source, threat model, test reports, artifact digest, failure exercise, dashboard, cleanup proof, and retrospective.

8. Build Reproducibility and Cache Security Lab

Compare clean builds across runners, design safe cache scopes, simulate cache poisoning, and document containment.

- Required evidence: architecture, pipeline source, threat model, test reports, artifact digest, failure exercise, dashboard, cleanup proof, and retrospective.

9. Pipeline Disaster-Recovery Game Day

Simulate runner fleet loss, registry outage, identity provider failure, and degraded emergency release with a second operator.

- Required evidence: architecture, pipeline source, threat model, test reports, artifact digest, failure exercise, dashboard, cleanup proof, and retrospective.

10. Compliance Evidence Manifest Generator

Produce a signed or digested manifest linking review, tests, policy, artifact, approval, deployment, verification, and recovery evidence.

- Required evidence: architecture, pipeline source, threat model, test reports, artifact digest, failure exercise, dashboard, cleanup proof, and retrospective.

Appendix E: Senior Interview Drill Bank

1. Explain continuous integration, continuous delivery, and continuous deployment using one release example.
2. How would you reduce a two-hour pipeline without reducing confidence?

3. How do you prevent a pull request from stealing production credentials?
4. Why should an organization build once and promote by digest?
5. How would you choose between rolling, blue-green, and canary deployment?
6. What evidence would you require before production approval?
7. How do you detect and manage flaky tests?
8. How would you recover from a registry outage during an incident?
9. What metrics diagnose slow delivery without ranking developers?
10. How do you make database changes compatible with rollback?
11. What are the risks of persistent self-hosted runners?
12. How do semantic versions and content digests complement each other?
13. What makes a reusable pipeline component safe for hundreds of repositories?
14. How would you prove that production is running the artifact built from a specific commit?
15. Describe a CI/CD failure you would intentionally inject before go-live.

Appendix F: Glossary

Term	Definition
Artifact	Immutable package or content produced by a build and promoted through environments.
Build provenance	Metadata describing where, when, how, and from which inputs an artifact was produced.
Canary release	Progressive exposure of a new version to a limited cohort before wider rollout.
Change fail percentage	Share of production changes that cause degraded service and require remediation.
Change lead time	Elapsed time for a change to move from version control commit to production.
Continuous delivery	Practice of keeping every accepted change deployable through automated build, test, and packaging.
Continuous deployment	Automatic production release of every change that satisfies all controls.
Continuous integration	Frequent integration of small changes into a shared main line with automated feedback.
Deployment frequency	How often production deployments occur over a defined period.
Digest	Cryptographic identifier for exact artifact content.
Failed deployment recovery time	Time required to recover from a failed deployment needing immediate intervention.

Term	Definition
Feature flag	Runtime control that changes behavior or exposure independently of code deployment.
Flaky test	Test that yields inconsistent results without a relevant change to the system under test.
GitOps	Principle-led operating model using declarative desired state, versioned and immutable storage, automated pull, and continuous reconciliation.
Hermetic build	Build whose inputs are explicitly declared and isolated from uncontrolled external state.
Pipeline	Automated workflow that transforms source and evidence into a qualified and deployed release.
Promotion	Controlled transition of the same immutable artifact to a higher trust or release state.
Runner	Compute environment that executes pipeline jobs.
Semantic versioning	Versioning contract using major, minor, and patch changes for a declared public API.
SBOM	Software bill of materials listing components included in an artifact.
SLO	Service level objective defining a measurable reliability target.
Workload identity	Short-lived identity issued to a workload based on trusted runtime claims.

Appendix G: Official Documentation and Standards Index

Confirm current versions and platform behavior before production implementation. These links are official primary sources.

- DORA software delivery performance metrics: [Open official page](#)
- DORA continuous delivery capability: [Open official page](#)
- Git branching workflows: [Open official page](#)
- GitHub Actions overview: [Open official page](#)
- GitHub Actions workflow artifacts: [Open official page](#)
- GitHub Actions secrets: [Open official page](#)
- Semantic Versioning 2.0.0: [Open official page](#)
- Open Container Initiative: [Open official page](#)
- OCI Distribution Specification: [Open official page](#)
- Docker multi-stage builds: [Open official page](#)

- Docker build cache optimization: [Open official page](#)
- SLSA build requirements and provenance: [Open official page](#)
- SLSA build provenance specification: [Open official page](#)
- OpenGitOps principles: [Open official page](#)
- Kubernetes Deployments: [Open official page](#)

Volume 10A Completion Statement

Part A establishes the vendor-neutral delivery model used throughout Volume 10. The next parts will implement these controls in Jenkins, GitHub Actions, GitLab CI/CD, Kubernetes GitOps, progressive delivery, DevSecOps, enterprise governance, and final production projects.

- Complete the capstone before selecting a platform-specific implementation. Tool syntax cannot compensate for missing artifact identity, authorization, evidence, verification, or recovery.
- Keep an error journal containing failed pipelines, misleading metrics, flaky tests, broken deployments, and the engineering change made after each lesson.
- Revisit official documentation as pipeline platforms, actions, runner images, supply-chain standards, and delivery research evolve.